IWEICT 2026 제출 논문 해설서

Bank Layout Optimization for Zero-Skipping FPGA CNNs

논문을 처음부터 끝까지, 숫자 하나까지 이해하기 위한 안내서

대상 원고: 제출본 `IWEICT2026_SUBMISSION_EN_12.docx`(저장소 `bankperm/PAPER_EN.md`와 같은 본문)
이 문서의 모든 숫자는 논문 본문과 저장소의 결과 파일(`bankperm/*.json`)에서 왔다. 출처는 13장에 모았다.
그림 ①–⑥은 설명용으로 새로 그렸고(작은 가상의 숫자를 쓴 그림은 그렇다고 적었다), Fig. 1–3은 논문의 그림 그대로다.

차례

1. 배경: 무엇이 문제인가
2. 시스템: 호스트와 FPGA 코어
3. 창 하나의 사이클: 식 (1)
4. 어디서 이득이 가능한가: 식 (2)
5. 세 가지 배치 L1, L2, L3
6. 실험 설정
7. 결과
8. 왜 lockstep을 유지하나 (FIFO 질문)
9. 직관: 왜 skew가 이기나
10. 한계와 미리 알던 것
11. 예상 질문과 답
12. 기호표와 용어집
13. 숫자의 출처

0. 한 쪽 요약

문제

0인 활성값을 건너뛰는 CNN 가속기는 합성곱의 한 창(window)을 여러 **뱅크**에 나눠 병렬로 처리한다. 그런데 뱅크들은 **함께(lockstep)** 다음 창으로 넘어가므로, 창마다 0이 아닌 값이 가장 많은 뱅크를 나머지가 기다린다. 이 대기가 0 건너뛰기로 얻은 이득을 깎아 먹는다.

아이디어

우리 시스템에서는 호스트(PC)가 창을 만들어 FPGA에 보낸다. 그러니 창의 각 탭(값 하나)을 **어느 슬롯에 넣을지**, 즉 어느 뱅크로 보낼지를 호스트가 소프트웨어로 정할 수 있다. 탭의 배치(layout)를 바꾸면 사이클 수는 바뀌지만 **FPGA 비트스트림도, 정수 출력도 그대로**다.

한 일

- **모양 조건 (2):** 층의 모양(K, COUT, W)만 보고 "여기서는 어떤 배치도 소용없다"를 판정하는 식.
- **세 배치:** 활성값 통계로 맞춘 **채널 재배치 L1**(가중치에 접어 넣음), 데이터가 필요 없는 **skew L2**(채널마다 탭을 네 뱅크에 흩음), 둘을 합친 **L3**.
- **검증:** 실제 보드(Zynq XC7Z020)에서 실행하기 **전에** 층마다 사이클을 예측해 커밋하고, 두 번의 사전등록(pre-registration) 연구로 가설을 미리 고정했다.

결과	숫자	뜻
skew L2 (데이터 없음)	9.20%	보드 망, 테스트 영상 79장, W = 4의 코어 사이클 절감(count)
skew + 재배치 L3	9.93%	같은 조건. 재배치 L1은 7.46%, 더 공들인 재배치 L1′은 8.10%
완벽 균형(상한)	12.02%	lockstep에서 어떤 배치로도 넘을 수 없는 값. L2는 그 76.6%, L3는 82.6%
보드 측정, 4장 합산	8.13% / 8.80%	L2 / L3. 네 영상 모두 L3 < L2 < L1′ < L1 < L0 (사이클)
사전 예측 적중	176 / 180	층 단위로 1사이클까지 정확. 나머지 4층도 +1사이클. 처음 돌린 L2·L3는 72층 중 71층
열 개 망	9 / 9	채널 수가 4의 배수인 9개 망 모두 L2 > L1, L3 > L2 (사전등록 가설 H5, H6)
CIFAR-10 분류기	3.47 / 8.54 / 9.10%	L1 / L2 / L3 (사전등록 밖). 같은 순서
FIFO로 뱅크를 분리하면?	최대 6.5%	L0의 뱅크 규칙을 유지하면 버퍼가 무한해도 이 이상 못 줄인다(계산한 상한). skew는 9.2%

읽는 순서. 1–2장은 배경과 하드웨어, 3–4장은 사이클을 세는 식, 5장은 세 배치, 6–7장은 실험과 결과, 8–9장은 "왜?"에 대한 답, 10장은 한계, 11장은 예상 질문과 답이다. 기호가 헷갈리면 12장의 기호표를 보면 된다.

1. 배경: 무엇이 문제인가

1.1 합성곱을 "창"으로 계산하기 (im2col)

3×3 합성곱에서 출력 픽셀 하나는 입력의 3×3 이웃 9곳(위치 **s** = 0…8)과, 각 위치의 입력 채널 **N**개(채널 **p** = 0…N−1)를 본다. 그러니 출력 픽셀 하나에 필요한 입력값은 **K = 9N**개이고, 이 K개를 한 줄로 펼친 것을 **창(window)**, 그 안의 값 하나하나를 **탭(tap)**이라 부른다. 출력 채널 **COUT**개 각각에 대해, 창의 K개 탭과 그 출력 채널의 가중치 K개를 곱해 더하면(내적) 출력값이 된다.

창을 펼치는 작업을 **im2col**이라 한다. 이 논문의 시스템에서는 호스트(PC)가 im2col을 해서 창을 FPGA로 보낸다. 탭의 기본 순서는 (ky, kx, 채널) 순, 즉 위치 s의 채널 p가 **슬롯 j = s·N + p**에 들어간다.

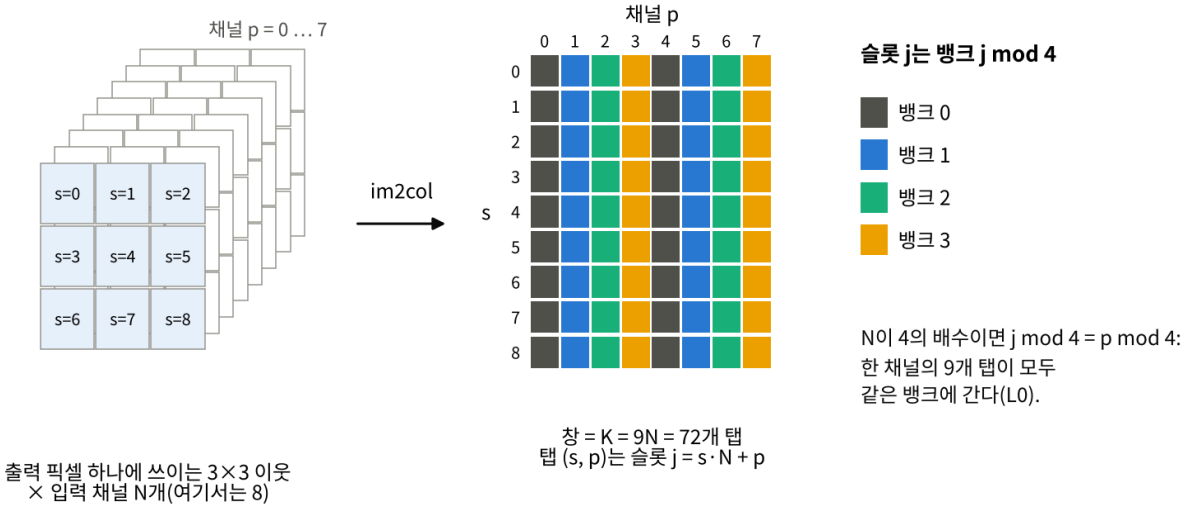


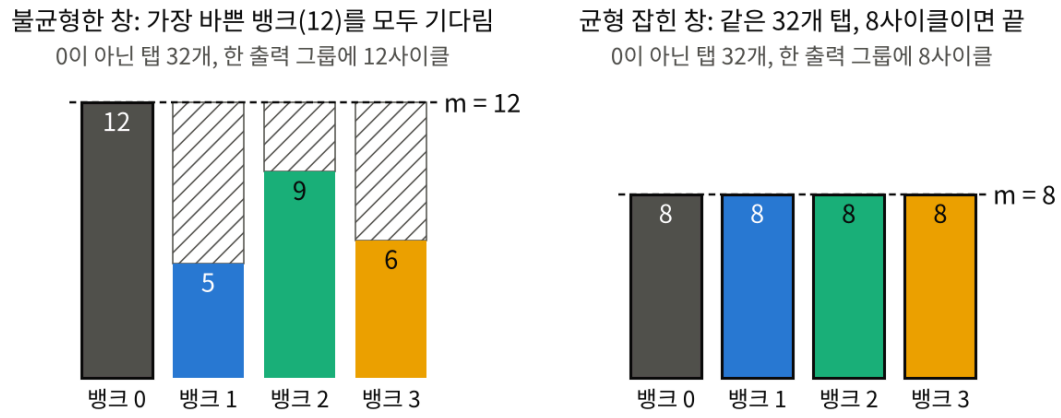
그림 ① im2col. 3×3 이웃 × 채널 8개(설명용 N = 8)가 72개 탭의 창이 된다. 색은 그 슬롯이 가는 뱅크(j mod 4)다. N이 4의 배수이면 j mod 4 = p mod 4이므로, 한 채널(한 열)의 9개 탭이 모두 같은 색, 같은 뱅크다.

1.2 0 건너뛰기 (zero skipping)

ReLU를 거친 활성값에는 0이 많다. 0을 곱하면 0이므로 그 곱셈은 건너뛰어도 결과가 같다. 이 논문의 보드 망(R1)에서는 숨은 층 창 값의 **37–55%만 0이 아니다**. 0인 탭을 건너뛰면 곱셈과 시간이 그만큼 줄어든다. 이것이 0 건너뛰기 가속기의 기본 아이디어다(Cnvlutin [1] 등).

1.3 뱅크와 lockstep: 가장 바쁜 뱅크를 기다린다

한 창을 빨리 처리하려고 탭을 **T = 4개의 뱅크**로 나눈다. 각 뱅크는 자기에게 온 탭 중 0이 아닌 것만 처리한다. 문제는 네 뱅크가 **함께(lockstep)** 움직인다는 점이다. 네 뱅크가 모두 끝나야 다음 창(정확히는 다음 출력 그룹)으로 넘어가므로, 창 하나의 계산 시간은 0이 아닌 탭이 **가장 많은 뱅크의 탭 수 m**이 정한다. 다른 뱅크는 그동안 논다.



빗금: 그 뱅크가 놀고 있는 사이클. 네 뱅크는 함께(lockstep) 다음 창으로 넘어간다.

그림 ② lockstep의 대가(설명용 숫자). 두 창 모두 0이 아닌 탭은 32개다. 왼쪽처럼 12/5/9/6으로 나뉘면 12사이클, 오른쪽처럼 8/8/8/8이면 8사이클이 걸린다. 같은 일을 하는데 배치만으로 1.5배 차이가 난다.

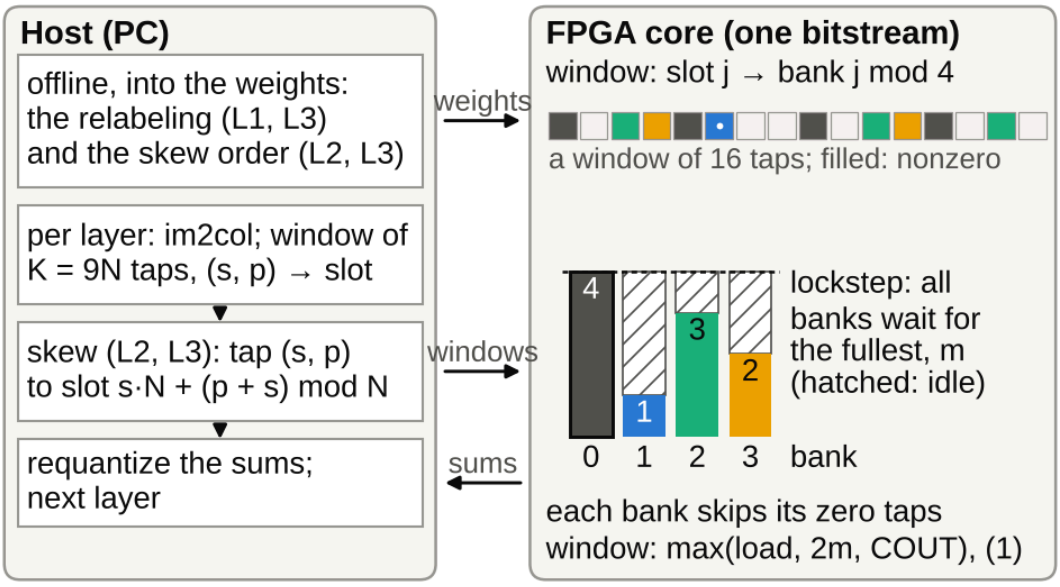
Cnvlutin [1]은 이 대기를 따로 집계할 만큼 알려진 문제로 다뤘다. 이 논문의 질문은 이것이다: **"탭을 뱅크에 어떻게 나눠 줄지를 바꿔서, m을 줄일 수 있을까? 그것도 하드웨어와 망을 건드리지 않고."**

1.4 기존 해법과 이 논문의 자리

접근	예	무엇을 바꾸나	이 논문과의 차이
망을 바꾼다	S2TA [2]	블록마다 0이 아닌 값의 수를 제한하도록 학습	망을 다시 학습해야 함. 우리는 학습된 망 그대로
하드웨어를 더한다	Sense [3]	실행 중에 채널을 다시 묶어 균형	Sense는 입력 맵이 "오프라인 학습으로 균형 맞출 수 없다"고 봄. 우리는 오프라인 배치로 상당 부분 줄일 수 있음을 보임
오프라인 재배열 (주로 가중치)	SparTen [4], Pool & Yu [5]	필터를 밀도로 정렬 / N:M 가지치기용 채널 순열을 앞 층에 접어 넣음	"순열을 앞 층에 접는다"는 기법은 같지만 목적(활성값 뱅크 균형)이 다름
입력 순열 + 인덱스	VARA [6] (ReRAM)	학습 데이터의 활성값으로 입력 순서를 정해 0인 것끼리 모아 통째로 건너뛸 블록을 만듦. 실행 중 인덱스 유닛으로 읽음	VARA는 0을 **모으고** 인덱스를 씀. 우리는 0 아닌 값을 **흩고**, 인덱스 없이 가중치에 접음
메모리 배치	Budnik & Kuck [7] (1971)	행렬의 행과 열을 충돌 없이 병렬로 읽는 skewed storage	그 skew를 창의 탭에 적용한 것이 L2

논문의 새로움 주장은 좁고 정확하다: "lockstep 0 건너뛰기 뱅크에 함께 켜지는(co-active) 채널을 정확하게(출력이 같게), 가중치에 접어 넣는 재배치로 흩는 연구는 찾지 못했다." 그리고 그 재배치를 데이터 없는 skew와 비교한 것, 모양 조건, 보드에서의 사전 예측이 기여다.

2. 시스템: 호스트와 FPGA 코어



논문 Fig. 1. 왼쪽 호스트는 배치를 오프라인에 가중치로 접어 넣고(재배치와 skew의 가중치 열 순서), 층마다 im2col로 창을 만들고, skew일 때는 탭을 skew 순서로 슬롯에 넣고, 결과를 재양자화한다. 오른쪽 코어는 비트스트림 하나로 모든 배치를 처리한다. 예시 창의 뱅크별 0 아닌 탭 수 4/1/3/2에서 lockstep 때문에 모두 4를 기다린다.

2.1 역할 분담

- **호스트(PC):** 층마다 창을 만들고(im2col), 층의 가중치와 함께 코어로 보내고, 층 사이에서 결과를 재양자화(requantize: 32비트 합을 다음 층의 7비트 입력으로 줄이는 것)한다.
- **코어(FPGA):** 모든 합성곱을 계산한다. 가중치는 INT8, 활성값은 7비트 부호 없는 정수(0–127), 누산은 INT32, 그리고 바이어스와 ReLU까지 코어 안에서 한다.

2.2 코어 안의 구조

- **곱셈기 64개 = P × T = 16 × 4.** P = 16은 **출력 레인**(한 번에 계산하는 출력 채널 16개), T = 4는 **탭 뱅크**다. 매 사이클 각 뱅크가 탭 하나를 내면, 그 탭이 16개 레인 모두에서 각 레인의 가중치와 곱해진다.
- **창 포트 W:** 사이클당 창의 값 W개를 받는다. 보드의 비트스트림은 W = 4다. W = 2는 RTL 시뮬레이션에서만 돌렸고, W = 1, 2, 4 모두 호스트 에뮬레이션 count로 평가했다.
- **슬롯 j → 뱅크 j mod 4.** 창의 j번째 슬롯 값은 뱅크 j mod 4로 간다. 하드웨어에 고정된 규칙이다.
- **sparse 모드(논문 전체에서 사용):** 각 뱅크는 0이 아닌 탭만, 뱅크 안에서의 번호(그 탭의 가중치를 찾는 주소)와 함께 저장한다. 창 버퍼가 두 개라, 다음 창을 적재하는 동안 현재 창을 계산한다(더블 버퍼링).
- **이슈:** 이슈 사이클마다 탭이 남은 뱅크는 탭 하나를 P개 레인 모두에 보낸다. 레인마다 네 뱅크의 곱을 덧셈 트리 하나로 더해 **누산기 하나**에 쌓는다. 이슈 카운터는 하나뿐이고 가장 바쁜 뱅크의 마지막 탭까지 돈다. 이것이 lockstep의 하드웨어적 이유다(8장).

> **RTL에서 확인한 것**
>
> `rtl/banked_window_mac.sv`: 뱅크마다 tuple RAM에 {뱅크 안 번호, 활성값}을 창 버퍼 두 개 분량 저장, 하나뿐인 `issue_pos`가 가장 바쁜 뱅크의 개수(`win_max`)까지 증가, 탭이 떨어진 뱅크는 곱을 0으로, 레인마다 T개 곱의 합 `psum`이 하나의 `accum`으로.

2.3 왜 "소프트웨어 선택"인가

뱅크는 슬롯 번호가 정하고, 슬롯 번호는 호스트가 정한다. 그러니 호스트가 탭을 다른 슬롯에 넣으면(그리고 그 탭에 곱할 가중치도 같은 자리로 옮기면) 같은 비트스트림, 같은 결과로 사이클만 달라진다. 이 "어느 탭을 어느 슬롯에 둘까"가 이 논문의 **배치(layout)**다.

3. 창 하나의 사이클: 식 (1)

$$C = \max(\lceil K/W \rceil + 1,\ \lceil COUT/P \rceil \cdot \max(1, m),\ COUT) \tag{1}$$

⌈x⌉는 올림이다. 세 항은 코어가 겹쳐서(파이프라인으로) 하는 세 가지 일이다. 겹쳐 진행되므로 창 하나의 시간은 셋 중 가장 긴 것이다.

항	뜻	보드 망 숨은 층 (K = 288, COUT = 32, W = 4)
⌈K/W⌉ + 1	**적재(load)**: 다음 창의 K개 값을 W개씩 받는 시간 + 시작 1사이클	⌈288/4⌉ + 1 = **73**
⌈COUT/P⌉ · max(1, m)	**계산**: 출력을 P = 16개씩 묶은 그룹 G = ⌈COUT/P⌉개를 차례로, 그룹마다 m사이클(가장 바쁜 뱅크). 탭이 모두 0이어도 그룹당 1사이클	G = 2이므로 **2m**
COUT	**내보내기**: 결과를 사이클당 1개씩	**32**

W = 4에서 보드 망의 숨은 층은 m > 36.5일 때(2m > 73) **계산이 창의 시간을 정한다(compute-bound)**. 이때만 m을 줄이는 배치가 사이클을 줄인다. m ≤ 36이면 적재가 더 길어서 m을 줄여도 소용없다.

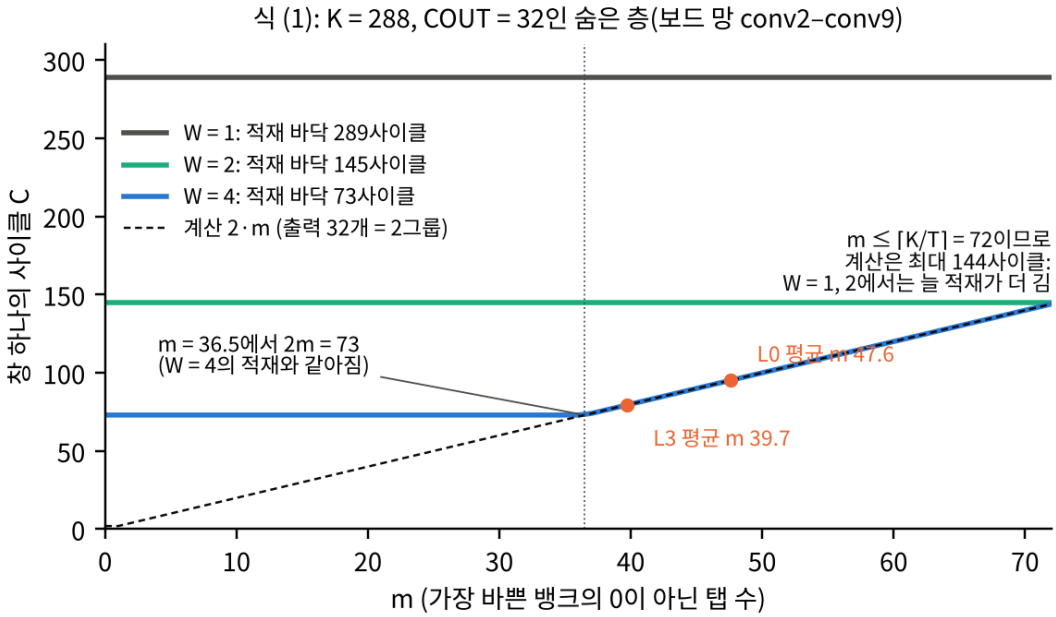


그림 ③ 식 (1)을 그린 것. 가로축 m, 세로축 창 하나의 사이클. W = 4(파랑)에서는 m이 36.5를 넘으면 계산 직선이 바닥(73)을 뚫고 올라가 m이 사이클을 정한다. W = 1, 2에서는 m이 최대치 72일 때도 계산(144)이 적재(289, 145)보다 짧아 배치가 아무 영향이 없다. conv5 창들의 평균 m은 L0 47.6, L3 39.7이다.

예: 논문 Fig. 2의 창 (보드 망 conv5, Set12 05.png)

0이 아닌 탭 156개, L0에서 뱅크별 43/**48**/29/36 → m = 48, 2m = 96사이클(적재 73보다 큼). L1에서는 41/**44**/31/40 → 88, L2에서는 36/**42**/40/38 → 84, L3에서는 36/40/38/**42** → 84. 완벽하게 나누면 ⌈156/4⌉ = 39 → 78사이클이다.

3.1 count와 예측 사이클

- **count(사이클 count):** 식 (1)을 한 층의 모든 창에 대해 더한 값. 호스트 에뮬레이션으로 계산한다(보드 없이 PC에서 같은 정수 연산을 돌리며 창마다 m을 센다). 보드 망은 2×2 space-to-depth로 해상도를 절반으로 줄이므로, Set12 05.png 같은 256×256 영상에서는 층마다 창이 128 × 128 = 16,384개다.
- **채움/비움(fill and drain):** 보드에서는 층마다 시작과 끝에 짧은 추가 시간이 있다. min(⌈K/W⌉+1, 마지막 창의 계산) + 3 + g 사이클 (g: 마지막 출력 그룹의 출력 수, COUT = 32면 16). 첫 층 conv1은 측정값 17사이클. 이 규칙은 네 망을 Set12 05.png로 보드에서 돌린 결과에서 읽어냈다.
- **예측 사이클 = count + 채움/비움.** 보드 실행 전에 이 값을 층마다 계산해 커밋했다(7.1절).

왜 count를 믿을 수 있나

같은 영상에서는 count로 잰 절감과 보드에서 잰 절감이 사실상 같다. 05.png의 L2: count 13,288,456 → 12,139,748, 보드 13,289,030 → 12,140,286, 둘 다 **8.644%**. 차이는 층마다 수백 사이클의 채움/비움뿐이다. 그래서 79장, 열 개 망의 결과를 count로 낸다.

4. 어디서 이득이 가능한가: 식 (2)와 완벽 균형

한 뱅크가 받을 수 있는 탭은 많아야 ⌈K/T⌉개다(K개 슬롯을 4개 뱅크가 나눠 가짐). 그러니 m ≤ ⌈K/T⌉이고 계산 항은 많아야 ⌈COUT/P⌉·⌈K/T⌉이다. 이것이 다른 두 항의 바닥보다 작거나 같으면, **어떤 입력에서도 어떤 창도 계산이 시간을 정하지 못하고**, 따라서 어떤 배치도 count를 바꿀 수 없다:

$$\lceil \mathrm{COUT}/P \rceil \cdot \lceil K/T \rceil \le \max(\, \lceil K/W \rceil + 1,\ \mathrm{COUT}\,) \qquad (2)$$

즉 **(2)가 깨지는 것은 이득의 필요조건**이다(충분조건은 아니다: (2)가 깨져도 실제 입력에서 m이 충분히 커야 이득이 난다). 올림을 빼고 정리하면 (2)는 **COUT ≤ P·T/W**, 즉 W = 4이면 COUT ≤ 16, W = 2이면 32, W = 1이면 64다. 층의 모양(K, COUT)과 코어(P, T, W)만으로, 데이터 없이 판정된다.

층 (예)	K	COUT	계산 최대	W = 1 바닥	W = 2 바닥	W = 4 바닥	이득 가능
32출력 숨은 층 (A, R1–R3 등)	288	32	2·72 = 144	289	145	73	W = 4만
64출력 숨은 층 (B)	576	64	4·144 = 576	577	289	145	W = 2, 4
45출력 숨은 층 (B-45)	405	45	3·102 = 306	406	204	103	W = 2, 4
R1의 마지막 층 conv10	324	4	1·81 = 81	325	163	82	없음
R1의 첫 층 conv1	36	32	2·9 = 18	37	32	32	없음

바닥 = max(⌈K/W⌉+1, COUT). 계산 최대 > 바닥일 때만 이득이 가능하다. 32출력 층은 W = 2에서 144 대 145로 아슬아슬하게 이득이 불가능하다.

이 표가 논문 결과의 모양을 미리 말해 준다. 디노이저의 32출력 층은 **W = 4에서만** 배치가 의미 있고, 마지막 층(COUT ≤ 4)은 어떤 W에서도 안 된다. 그래서 배치는 conv2부터 conv(L−1)까지의 입력에만 적용한다. 실제로 (2)가 성립하는 모든 곳에서 count가 L0와 정확히 같았다(사전등록 가설 H1, H4).

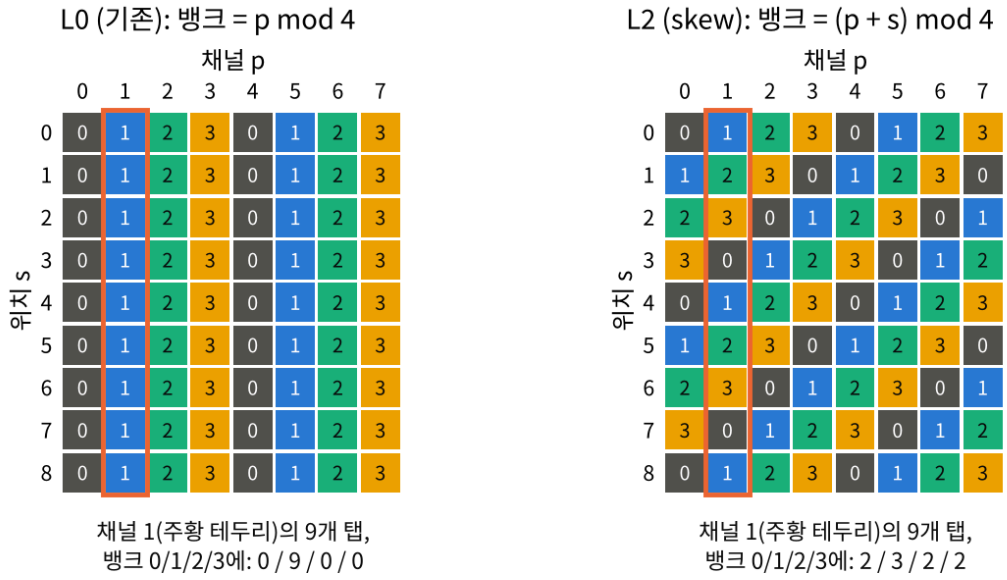
4.1 완벽 균형: lockstep 배치의 상한

창 하나에 0이 아닌 탭이 n개면, 4개 뱅크에 아무리 잘 나눠도 m은 ⌈n/4⌉보다 작아질 수 없다. 이 최선의 경우를 **완벽 균형 (perfect balance)**이라 한다. 창마다 데이터를 보고 따로 나눠도 마찬가지이므로, lockstep 코어에서 **어떤 배치도 완벽 균형보다 나을 수 없다**. 보드 망에서 완벽 균형의 절감은 12.02%이고, 7.3절의 "share"는 각 배치가 이 상한의 몇 %를 얻었는지다.

5. 세 가지 배치 L1, L2, L3

5.1 L0: 기존 배치와 그 약점

기존 순서에서 탭 (s, p)는 슬롯 s·N + p에 있고, N이 4의 배수이면 뱅크는 **p mod 4**다. 즉 각 뱅크가 N/4개 채널을 **통째로**(9개 위치 모두) 맡는다. 어떤 채널이 자주 0이 아니면(예: 밝은 영역에 잘 반응하는 채널), 그 채널의 9개 탭이 모두 한 뱅크로 가고, 그런 채널이 같은 뱅크에 몰리면 그 뱅크가 늘 바쁘다.



칸 안의 숫자는 그 탭의 뱅크. 자주 켜지는 채널이 있으면 L0에서는 그 채널의 탭이 한 뱅크에 몰리고, L2에서는 3/2/2/2로 흩어진다.

그림 ④ L0와 L2의 뱅크 지도(설명용 N = 8). 칸의 숫자가 그 탭의 뱅크다. 주황 테두리의 채널 1은 L0에서 9개 탭이 모두 뱅크 1로, L2에서는 뱅크 0/1/2/3에 2/3/2/2개로 흩어진다.

5.2 L1: 채널 재배치(relabeling), 가중치에 접어 넣기

π를 "숨은 맵의 위치 p에 원래 채널 π(p)가 온다"는 순열이라 하자. 앞 층(**생산자**)이 출력 채널을 π 순서로 계산하도록 가중치 행, 바이어스, 재양자화 스케일을 재배열하고, 다음 층(**소비자**)의 가중치 열도 맞춘다(탭 s·N + p의 가중치는 채널 π(p)의 것). 모두 오프라인에 한 번 하는 일이고, 실행 중에는 추가 작업이 없다.

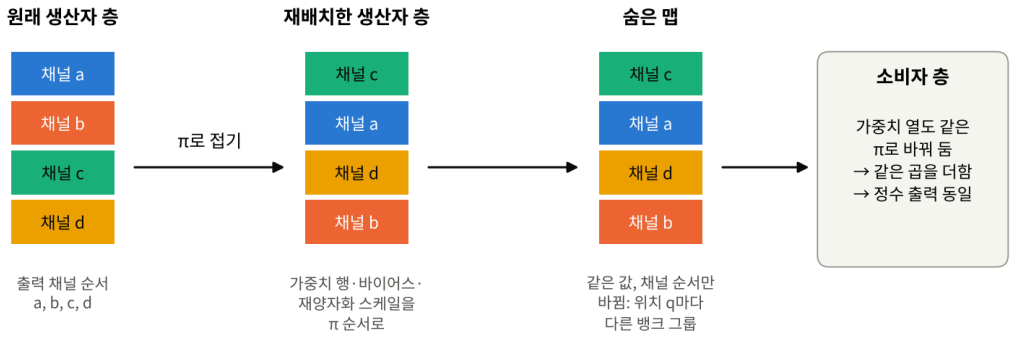


그림 ⑤ 재배치를 접어 넣기(설명용 채널 4개). 생산자가 채널을 c, a, d, b 순서로 내놓게 하고, 소비자의 가중치 열도 같은 순서로 바꿔 두면, 소비자는 같은 곱들을 더하므로 결과가 같다. 바뀌는 것은 어느 채널이 어느 위치(따라서 어느 뱅크 묶음)에 있는가뿐이다.

왜 출력이 정확히 같은가. ReLU와 재양자화는 채널마다 따로 작용하므로 채널 순서를 바꿔도 각 채널의 값은 같다. 소비자는 같은 INT32 곱들을 순서만 바꿔 더한다. 그래서 **모든 입력에서 정수 출력이 같다**. 어떤 채널끼리 같은 뱅크를 쓰는지(class = 채널 자리 p mod 4)는 π가 정한다.

- L층 망에서 conv1부터 conv(L−2)까지의 출력을 재배치한다. 마지막 층은 이득이 불가능하므로(4장) 입력을 그대로 둔다.
- maxout·평균 망(두 맵 j와 j+N의 최대 또는 평균)에서는 짝인 두 맵이 함께 움직인다.
- **한계:** 채널 단위로만 바꾼다. 한 채널의 9개 탭은 여전히 같은 뱅크에 있다.

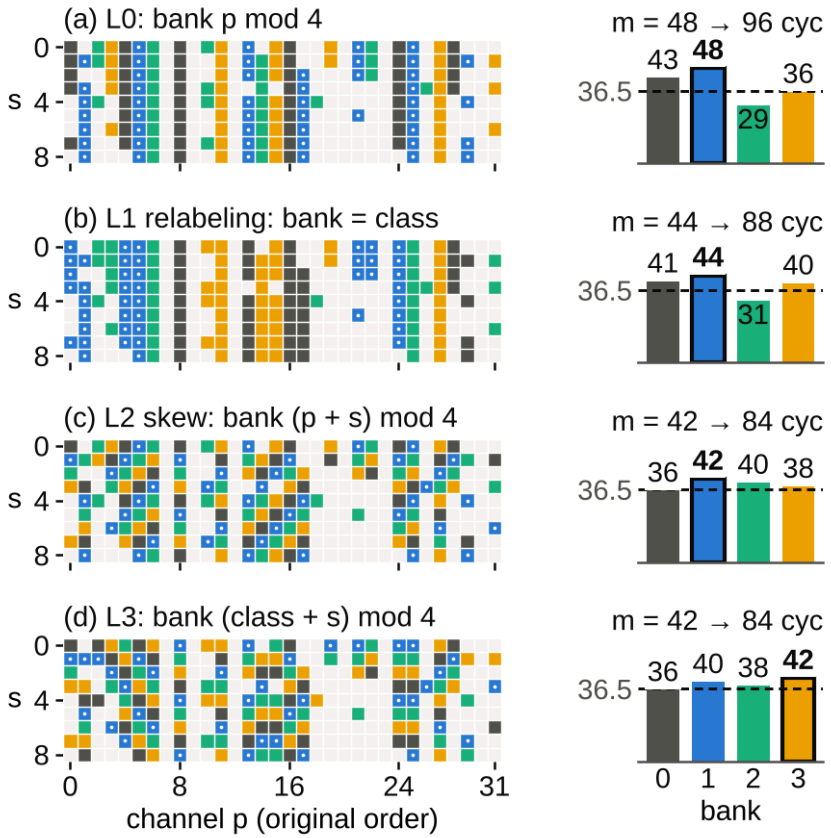
5.3 L2: skew (데이터가 필요 없다)

Budnik & Kuck의 skewed storage [7]를 탭에 적용한다. N이 4의 배수일 때, 호스트가 위치 s의 채널 p를 **슬롯 s·N + (p + s) mod N**에 넣는다. 그러면 뱅크는 **(p + s) mod 4**가 되어, 위치가 하나 바뀔 때마다 같은 채널의 뱅크가 하나씩 밀린다. 9개 위치에서 (p + s) mod 4는 p의 원래 뱅크를 3번(s = 0, 4, 8), 나머지를 2번씩 지나므로, **한 채널의 9개 탭이 3/2/2/2로 네 뱅크에 흩어진다**(그림 ④ 오른쪽).

- 창의 탭과 소비자 가중치 열을 같은 순열로 바꾸므로 각 출력은 같은 곱들의 합이다.
- **정수 결과가 같은 이유:** 정수 덧셈은 순서를 바꿔도 결과가 같다. 넘침(overflow)만 없으면 된다. 어떤 순서로 더해도 부분합은 $|\text{bias}| + K \cdot 127 \cdot 128 < 2^{31}$을 넘지 않으므로 INT32 결과가 같다.
- 데이터도, 맞추기(fitting)도, RTL 변경도 필요 없다. 호스트 비용은 im2col 뒤 재배열 4.2 ms(05.png 기준)뿐.
- **N mod 4 = 1인 B-45(채널 45개):** 기존 순서에서 이미 j mod 4 = (45s + p) mod 4 = (s + p) mod 4이므로 L0가 곧 skew다. 그래서 B-45에서는 L2 = L0, L3 = L1이다.

5.4 L3: skew 위에서 다시 재배치

skew 아래에서 뱅크 b는 위치 s에서 class (b − s) mod 4의 채널을 담는다(class = p mod 4). 9개 위치를 지나며 class b는 위치 0, 4, 8에서 세 번, 나머지 class는 두 번씩 뱅크 b에 온다. 그러니 skew는 어떤 class가 0 아닌 값을 많이 가질 때 그 과잉을 **희석할 뿐 없애지는 못한다**. 그리고 어떤 채널들이 한 class를 이루는지는 여전히 자유다. L3는 L2에서 출발해 이 skew 규칙 아래서 재배치를 다시 맞춘 뒤 skew한다(논문 Fig. 2d).



논문 Fig. 2. 보드 망 conv5의 실제 창 하나(Set12 05.png)를 네 배치로 본 것. 칸: 0이 아니면 뱅크 색. 막대: 뱅크별 0 아닌 탭 수, 굵은 테두리가 가장 바쁜 뱅크 m. 점선 m = 36.5는 2m이 적재 73과 같아지는 높이다. m: 48 → 44 → 42 → 42.

5.5 재배치를 맞추는 방법 (fitting)

- **층마다 따로:** 재배치된 창은 원래 창의 열을 바꾼 것일 뿐이라, 층마다 독립적으로 맞출 수 있다.
- **데이터:** 보정(calibration) 영상 한 장(Set12 01.png)에서 층마다 창 2048개를 뽑는다.
- **목적함수:** 그 창들의 식 (1) 평균(W = 4).
- **탐색:** 항등 순열에서 출발해, 서로 다른 class의 두 채널을 맞바꾸는 제안을 400번 하고, 목적함수가 줄 때만 받아들인다(탐욕적 국소 탐색). 원래 뱅크 규칙으로 하면 L1, skew 규칙으로 하면 L3.
- **L1′(보드 망 전용):** 더 공들인 재배치. Set12 01–03, 영상마다 잡음 두 번, 항등 순열과 무작위 시작점 4개에서 각각 탐색(층마다 5 × 401 = 2005번 평가). "재배치에 데이터와 탐색을 더 주면 skew를 이기나?"를 확인하려는 것이다 (결과: 8.10% < skew 9.20%).
- 망 하나를 맞추는 데 30초 미만(L1, L3). L2는 맞출 것이 없다.

5.6 정말 출력이 같은지 확인한 것

- 호스트 에뮬레이션: 모든 망의 L1이 테스트 영상 2장에서, 보드 망의 L1과 L1′이 80장 모두에서 원래 출력을 그대로 냈다.
- L2와 L3는 모든 망에서 Set12 05.png의 모든 층, 모든 창의 INT32 누산값이 같았다(L3는 채널 순서만 다름).
- 보드에서는 모든 층의 출력이 정수 모델과 일치했다.

6. 실험 설정

6.1 열 개의 망

모두 σ = 25 가우시안 잡음을 지우는 DnCNN 계열 정수 디노이저 [8]이고 입력은 7비트다. A를 뺀 모든 망은 양자화를 고려한 학습(QAT)을 거쳤다.

이름	구조	숨은 채널 N	비고
A	7층	32	QAT 없음
B	10층, 2×2 space-to-depth 입력	64	64출력 층이라 W = 2에서도 이득 가능
R1, R2, R3	B와 같은 구조, ReLU (학습 시드 s1–s3)	32	**R1 = 보드 망**, Set12 05.png에서 29.54 dB
M1, M2, M3	maxout: 64개를 계산해 짝마다 최대 (시드 s1–s3)	32	재배치 때 짝이 함께 이동
Av2	짝의 평균 (시드 s2)	32	
B-45	B와 같은 구조, ReLU	45	N mod 4 = 1: L2 = L0, L3 = L1

space-to-depth: 2×2 픽셀 블록을 채널 4개로 바꿔 해상도를 절반으로 줄이는 것. 보드 망 R1은 conv1(K = 36, 출력 32), conv2–conv9(K = 288, 출력 32), conv10(K = 324, 출력 4)의 10층이다.

6.2 영상과 통계

- **영상:** Set12(12장)와 BSD68(68장), 모두 80장, 잡음은 영상마다 한 번 뽑는다.
- **pooled(합산) 절감:** 보정 영상 Set12 01.png를 뺀 79장의 사이클을 모두 더한 뒤 계산한 절감률이다. 영상별 절감률의 평균이 아니다. 08.png처럼 큰 영상은 그만큼 무겁게 반영된다.
- **95% 구간:** paired bootstrap. 79장에서 영상을 복원추출로 다시 뽑아 합산 절감을 여러 번 계산한다. 같은 영상 묶음에서 두 배치를 비교하므로 "paired"다. 구간은 영상의 변동만 반영하고, 맞추기나 학습의 변동은 반영하지 않는다.
- **%p(퍼센트포인트):** 두 절감률의 차. 9.93% − 9.20% = 0.73%p.

6.3 사전등록(pre-registration)이란

실험하기 **전에** 무엇을 어떻게 하고 무엇을 기대하는지(레시피, 평가 방법, 가설)를 문서로 고정해 커밋해 두는 것이다. 결과를 본 뒤 유리하게 방법을 바꾸거나 좋은 결과만 고르는 일을 막는다. 이 논문에는 프로토콜이 두 개 있다.

프로토콜	대상	가설
`PROTOCOL.md` (1차)	재배치 L1, 열 개 망	**H1** (2)가 성립하는 곳에서 L1 count = L0 · **H2** W = 4에서 L1 절감 > 0 · **H3** W = 2에서 (2)가 허용하는 곳의 L1 절감 > 0
`PROTOCOL_2.md` (2차)	skew L2, 조합 L3	**H4** (2)가 성립하는 곳에서 L2, L3 count = L0 · **H5** W = 4에서 L2 > L1 · **H6** W = 4에서 L3 > L2

사전등록 전에 이미 알던 것이 있었고, 논문은 그것을 숨기지 않고 VI장에 밝혔다(이 문서 10장).

6.4 보드

- Zynq XC7Z020용 W = 4 비트스트림. 100 MHz에서 타이밍을 만족하고, LUT 11,151개(21%), BRAM36 환산 140개 중 71개, DSP 0개를 쓴다. 보드 프리셋인 50 MHz에서 돌렸다.

- 비트스트림 하나로 모든 배치를 돌린다. 드라이버는 층마다 출력을 정수 모델과 대조하고, 사이클 카운터 **CYCLES**와 DMA 대기 카운터 **IN_STALL**을 읽는다.
- 다섯 배치(L0, L1, L1′, L2, L3)를 Set12 05–08에서 한 번씩 돌렸다. conv2–conv10의 사이클은 실행 전에 예측해 커밋했다.
- 그 밖의 모든 사이클 숫자는 호스트 에뮬레이션 count다. 에뮬레이션은 모든 망의 기준 PSNR을 80장 모두에서 재현한다(같은 정수 연산이라는 확인).

7. 결과

7.1 보드 측정 (논문 Table I)

배치	05	06	07	08
L0 (10^6 사이클)	13.289	13.466	13.412	51.977
L1 재배치	7.02	7.91	7.22	5.93
L1′ 재배치	7.60	9.01	7.89	6.58
L2 skew	8.64	9.97	8.95	7.31
L3 skew + 재배치	9.39	10.68	9.63	7.95

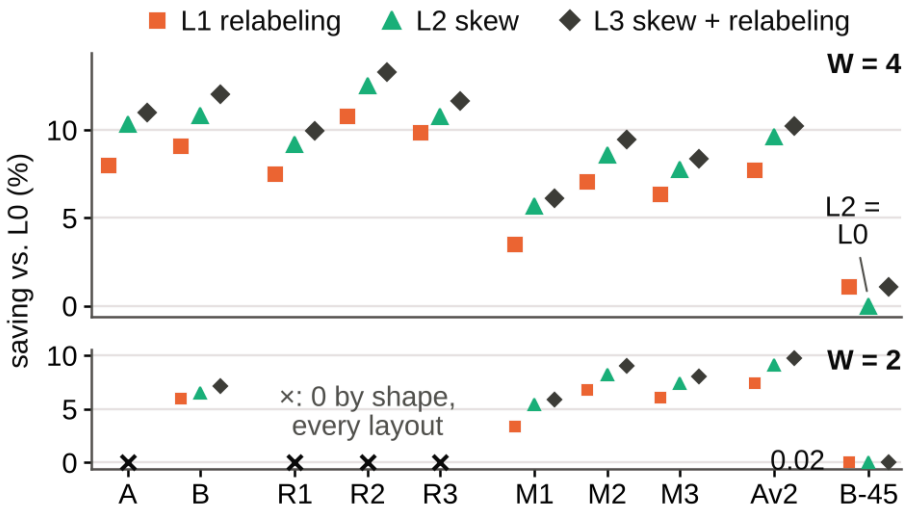
L0 줄은 총 사이클, 나머지는 L0 대비 절감(%). 08.png는 512×512, 나머지는 256×256이다.

- **순서:** 네 영상 모두 L3 < L2 < L1′ < L1 < L0(사이클), 예측과 같다. 네 장 합산 절감은 L2 8.13%, L3 8.80%.
- **층 단위 예측:** 180개 층(5배치 × 4영상 × conv2–conv10의 9층) 중 **176개**에서 CYCLES − IN_STALL이 예측과 정확히 같았다.
- **틀린 4개:** 모두 conv8이고 1사이클 많았다. 실행 후에 보니 정확히 "마지막 창의 계산이 30사이클"인 층들이었다(다른 conv2–conv9 층은 32–56). 채움/비움 규칙을 읽어낸 실행에는 없던 경우다. 예측은 고치지 않고 그대로 보고했다.
- **처음 돌린 배치:** L2와 L3는 보드에서 처음 돌렸는데 72층 중 71층이 정확했다. conv10은 어느 배치든 L0와 같은 창을 보므로 그것을 빼면 64층 중 63층이다.

> **이 일치가 증명하지 않는 것**
>
> 이 비트스트림의 IN_STALL은 창 경계에서 DMA가 데이터를 늦게 주는 경우를 세지 않는다. 그래서 모든 배치에 똑같이 생기는 정체는 채움/비움 규칙 안에 숨어 있을 수 있다. 예측의 일치가 배제하는 것은 "배치마다 다르게 생기는 정체"뿐이다.

보드 절감(8.13%)이 count(9.20%)보다 작은 이유는 영상 묶음이 다르기 때문이다(4장 대 79장, 4장 중에서는 큰 08.png의 절감이 가장 작다). 같은 영상에서는 count와 보드 절감이 같다(3.1절의 05.png 예).

7.2 열 개 망, 두 프로토콜 (논문 Fig. 3)



논문 Fig. 3. 79장 합산 절감, 위 W = 4, 아래 W = 2. 주황 네모 L1, 초록 세모 L2, 검은 마름모 L3. ×는 (2)가 모든 층에서 성립해 모든 배치가 L0와 같은 곳(W = 1은 모든 망이 그렇다). B-45에서 L2 = L0.

- **H1 성립:** (2)가 성립하는 곳(W = 1의 모든 망, W = 2의 A와 R1–R3)에서 재배치 count가 L0와 80장 모두 같았다.
- **H2 성립:** W = 4에서 L1 절감이 열 개 망 모두 양수, 1.09–10.80%.
- **H3 성립:** W = 2에서 (2)가 허용하는 여섯 망 모두 양수(다섯 망 3.36–7.37%, B-45 0.022%).
- **H4 성립:** (2)가 성립하는 곳에서 L2와 L3도 모든 층, 80장 모두 L0와 같았다.
- **H5 성립:** W = 4에서 N mod 4 = 0인 아홉 망 모두 L2가 L1보다 0.90–2.36%p 더 줄였다.
- **H6 성립:** 아홉 망 모두 L3가 L2보다 0.48–1.18%p 더 줄였다(R1에서 0.73, 95% 구간 [0.70, 0.75]).
- **가설 밖의 확인:** L3는 보정 영상 한 장으로만 맞췄는데도 711개 영상–망 쌍(9망 × 79장) 모두에서 L2보다 count가 작았고, 합산으로는 맞춘 69개 층 모두에서 작았다. W = 2에서도 이득이 가능한 다섯 망에서 L3 > L2 > L1 순.
- 열 개 망에서는 어떤 영상도, 어떤 W에서도 어떤 배치가 L0보다 count가 커진 적이 없다.

7.3 보드 망, 79장 (논문 Table II)

배치	맞춘 영상	합산 절감 (%) [95% 구간]	share (%)
밀도 균형 (SparTen식 [4])	1	3.69 [3.50, 3.88]	30.7
Sense식 정렬 [3]	각 테스트 영상	4.89	40.7
L1 재배치	1	7.46 [7.16, 7.78]	62.1
L1′ 재배치	3 †	8.10 [7.80, 8.39]	67.4
L2 skew	없음	9.20 [8.82, 9.58]	76.6
L3 skew + 재배치	1	9.93 [9.54, 10.32]	82.6
완벽 균형	—	12.02 [11.59, 12.44]	100

share = 그 배치의 절감 ÷ 완벽 균형의 절감. †: 맞춘 3장 중 2장(Set12 02, 03)이 79장에 들어 있다. 밀도·Sense식은 세 변형 중 가장 좋은 것.

- 79장 **각각에서** L2가 두 재배치보다, L3가 L2보다 많이 줄였다.

- 데이터가 필요 없는 L2가 완벽 균형 절감의 76.6%를, L3가 82.6%를 얻었다. L3는 L2가 남긴 불균형의 26%를 더 없앴다: (9.93 − 9.20) ÷ (12.02 − 9.20) ≈ 0.26. 호스트 비용은 L2와 같다(같은 재배열, 재배치는 가중치 안에).
- **가장 눈여겨볼 비교:** L1 레시피를 **각 테스트 영상 자신에** 맞춰도(실제로는 쓸 수 없는, 유리한 조건) 8.45%로, 데이터 없는 skew 9.20%에 못 미친다.
- Sense식(채널별 0 아닌 수로 정렬)과 SparTen식 밀도 균형은 더 적게 줄였다.

7.4 CIFAR-10 분류기 (사전등록 밖)

디노이저가 아닌 망에서도 같은지 보려고, 저장소의 `cifar/` 스크립트로 CIFAR-10 분류기(3×3 합성곱 6개, 채널 3→32→32→64→64→128→128)를 15 epoch 학습했다(정수 정확도 88.09%). 테스트 영상 0번 한 장으로 맞추고 나머지 9,999장에서 셌다. 숨은 입력은 모두(conv2–conv6) 배치를 바꿨다.

W = 4	L1	L2	L3	완벽 균형
합산 절감 (%)	3.47	8.54	9.10	13.08

- 순서가 디노이저와 같다. L2와 L3는 9,999장 모두에서 이득이었다.
- L1은 176장에서 L0보다 나빠졌다. 32×32 영상 한 장에서 나오는 창이 적어(1024/256/256/64/64개) 보정 영상에 과적합했기 때문이다(보정 영상에서는 8.22%).
- W = 1에서는 128출력 층(conv5, conv6)이 (2)를 깨서 이득이 가능해지는데, 세 배치 모두 일부 영상에서 아주 조금(0.4% 미만) 나빠졌다. 논문은 이것도 적었다.

7.5 비용과 전체 시간

- 맞추기: 망당 30초 미만(L1, L3). L2는 0.
- 호스트(x86 서버, 단일 스레드) 작업, 05.png: 입력·im2col·재양자화에 31–32 ms. 코어는 50 MHz에서 266 ms.
- 둘을 연달아 하면 L1′의 코어 절감 7.60%가 전체 시간으로는 6.3–6.8%다. 호스트가 다음 영상을 코어와 겹쳐 처리하면 7.60%를 다 얻는다. skew 재배열은 4.2 ms를 더한다.
- 창과 결과의 전송 시간은 이 계산에 없고, Zynq의 ARM 코어는 서버보다 느릴 것이다.

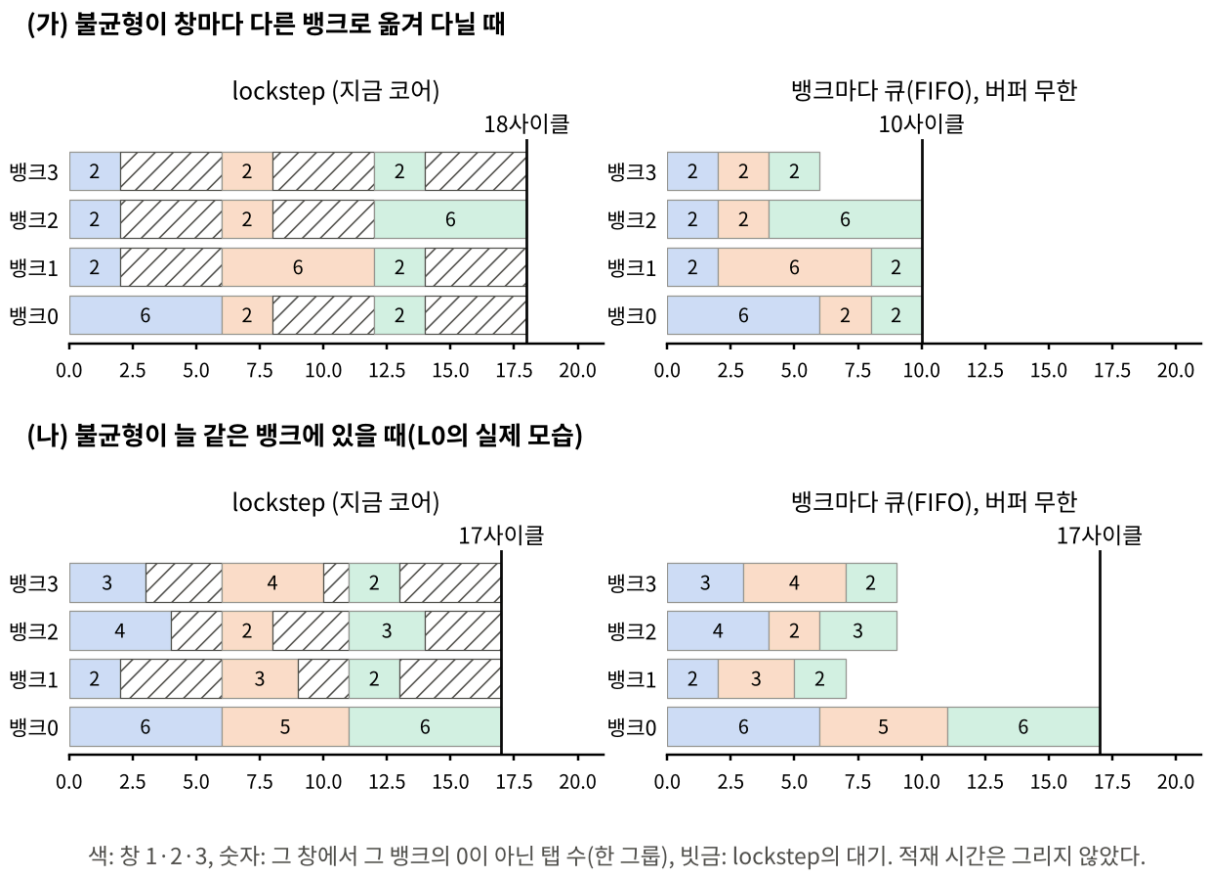
8. 왜 lockstep을 유지하나: "FIFO를 쓰면 되지 않나?"

가장 예상되는 반론은 이것이다: "lockstep이 문제라면 뱅크마다 큐(FIFO)를 두어 빨리 끝난 뱅크가 다음 창을 먼저 하게 하면 되지 않나? 이 문제는 스스로 만든 것 아닌가?" 논문(Sec. II 끝 "Why lockstep")의 답은 두 부분이다.

8.1 비용

지금 코어는 레인마다 네 뱅크의 곱을 한 번에 더해 누산기 하나에 쌓는다. 뱅크가 서로 다른 창을 처리하게 하려면 뱅크·레인·앞서 간 창마다 부분합을 따로 저장해야 하고, 더 깊은 버퍼와 마지막 합산 단계가 필요하다.

8.2 효과: L0의 불균형은 창마다 바뀌지 않는다



색: 창 1·2·3, 숫자: 그 창에서 그 뱅크의 0이 아닌 탭 수(한 그룹), 빗금: lockstep의 대기. 적재 시간은 그리지 않았다.

그림 ⑥ FIFO가 도움이 되는 경우와 안 되는 경우(설명용 숫자, 창 3개). (가) 바쁜 뱅크가 창마다 바뀌면 FIFO로 서로 메워 18 → 10사이클. (나) 늘 같은 뱅크(0)가 바쁘면 FIFO가 있어도 그 뱅크의 총량 17사이클이 그대로 걸린다.

L0에서는 같은 채널이 늘 같은 뱅크에 있으므로, 자주 켜지는 채널을 가진 뱅크는 층 전체에서 계속 바쁘다(그림 ⑥의 (나)). 보드 망에서 한 층 전체를 놓고 보면 가장 바쁜 뱅크가 평균 뱅크의 최대 **1.35배**의 탭을 처리한다(conv9). skew를 적용한 층(conv2–conv9)에서는 1.04배 이하다.

그래서 저장소의 `decouple_bound.py`로 상한을 계산했다. 뱅크를 분리해도 뱅크 b는 자기 몫의 0 아닌 탭을 출력 그룹마다 한 번씩 처리해야 하므로, 층의 시간은 적어도 **max(N·max(적재, COUT), G·max$_b$ Σ$_w$ n$_{w,b}$)**다($n_{w,b}$: 창 w에서 뱅크 b의 0 아닌 탭 수). 이 하한으로 계산하면:

보드 망, 79장, L0 count 대비	절감
L0의 뱅크 규칙 + 뱅크 분리, 버퍼 무한 (상한)	최대 6.54%
같은 조건의 창 단위 시뮬레이션	6.00%
같은 조건, 지금처럼 창 버퍼 2개 (시뮬레이션)	0.64%
lockstep 그대로 + skew L2	**9.20%**
skew + 뱅크 분리 (상한 / 버퍼 3개 시뮬레이션)	12.96% / 12.15%

즉 **하드웨어를 바꿔(FIFO) 얻을 수 있는 것보다, 배치만 바꿔(skew) 얻는 것이 더 크다.** 반대로 skew와 뱅크 분리를 합치면 완벽 균형(12.02%)도 넘을 수 있다. 둘은 경쟁이 아니라 보완 관계이고, 이 논문의 주제는 "하드웨어를 바꾸지 않고 배치만으로"다.

> **흔한 오해: "12.02%가 모든 뱅크 변경의 천장"**
>
> 완벽 균형은 **창 하나 안에서** 가장 고르게 나눈 경우이므로 lockstep 배치의 상한이다. 뱅크를 분리하면 창들 사이에서도 일을 나눌 수 있어 이보다 나을 수 있다(위 표의 12.96%). 그래서 논문은 12.02%를 "lockstep 배치가 얻을 수 있는 최대"라고만 쓴다.

이 상한과 시뮬레이션은 보드나 RTL에서 만든 것이 아니라 창 데이터로 계산한 것이며, 두 프로토콜이 끝난 뒤에 계산했다(사전등록 밖). 2차 검증에서 별도 스크립트로 9장의 900개 값이 정확히 재현되었다.

9. 직관: 왜 skew가 재배치보다 잘 되고, 합치면 조금만 더 좋아지나

1. **L0의 불균형은 "채널 단위로 몰림"에서 온다.** 자주 켜지는 채널 하나의 9개 탭이 전부 한 뱅크로 간다.
2. **재배치(L1)는 "어느 채널끼리 한 뱅크"만 바꾼다.** 채널의 9개 탭은 여전히 한 뱅크에 묶여 있다. 그리고 π는 하나로 고정되어 모든 영상, 모든 위치에 같이 쓰인다. 어떤 채널이 자주 켜지는지는 영상마다, 영상의 영역마다 다르므로 고정된 묶음으로 맞추는 데 한계가 있다. 각 테스트 영상에 따로 맞춰도 8.45%에 그친다.
3. **skew(L2)는 채널 하나의 탭을 3/2/2/2로 흩는다.** 어떤 채널이 켜지든 그 일이 네 뱅크에 나뉜다. 그래서 데이터 없이, 영상에 상관없이 균형이 좋아진다.
4. **skew 뒤에 남는 불균형**은 같은 class(p mod 4)가 한 뱅크에 세 번 오는 데서 온다(5.4절). 한 class에 바쁜 채널이 몰려 있으면 여전히 치우친다. L3는 class 구성을 다시 맞춰 이 나머지를 줄인다. 하지만 skew가 이미 대부분을 흩었으므로 추가 이득은 작다(0.48–1.18%p, 남은 불균형의 26%). 논문이 "둘은 일부만 더해진다"고 쓴 이유다.
5. **왜 W = 4에서만 보이나.** W가 크면 적재가 빨라 계산(뱅크 불균형)이 창 시간을 정하는 일이 잦아진다. 32출력 층은 W = 4에서만 조건 (2)가 깨진다(4장).

10. 한계와 미리 알던 것

10.1 범위

- 보드: 망 하나(R1), 영상 네 장, 각 한 번 실행.
- count: σ = 25의 DnCNN 계열 디노이저 열 개(영상마다 잡음 한 번), 그리고 프로토콜 밖의 CIFAR-10 분류기 하나(15 epoch).
- 배치마다 맞추기는 한 번(구간은 영상만 다시 뽑는다). BSD68은 망을 개발할 때도 쓰였다.

- 뱅크 분리는 상한만 계산했고 만들지 않았다. 다른 P, T, 커널 크기, 에너지는 평가하지 않았다.

10.2 사전등록 전에 알던 것 (논문 VI장)

- 1차 전: R1에서 L1의 절감을 이미 셌고, 다른 재배치 탐색으로 B와 M1에서 W = 4 절감이 있다는 것을 알았다.
- 2차 전: L2를 R1에서(80장 모두), 다른 네 망에서 한 장씩, B와 M1에서는 여덟 장씩 셌고, L3를 R1의 일곱 장에서 셌다.
- L1′은 BSD68 count를 본 뒤 보드용으로 골랐다.
- L0, L1, L1′은 보드 영상에서 이미 돌린 적이 있다. 채움/비움 규칙은 L0를 포함한 05.png 실행에서 읽었고, 예측 코드는 세 배치의 05.png 총합을 재현해야만 결과를 쓰도록 되어 있었다. 그래서 05.png의 이 세 배치 27개 층(3 × 9)은 "표본 밖 예측"이 아니다.

이런 것을 굳이 적는 이유: 사전등록의 신뢰는 "무엇을 미리 알았는지"를 밝힐 때 생긴다. 숨기면 좋은 결과가 우연인지 선택인지 독자가 알 수 없다.

11. 예상 질문과 답

Q1. 결국 채널 순열 아닌가? 무엇이 새로운가?

채널 순열을 가중치에 접는 기법 자체는 Pool & Yu [5] 등에 있다. 새로운 것은 (1) 그것을 lockstep 0 건너뛰기 뱅크의 활성값 균형에, 출력이 정확히 같게 적용한 것, (2) 데이터 없는 skew가 그 재배치보다 낫다는 비교(아홉 망 모두, 영상마다), (3) 어디서 이득이 가능한지 모양만으로 정하는 조건 (2), (4) 보드에서 실행 전에 층 단위로 사이클을 예측해 맞힌 것이다.

Q2. 뱅크마다 FIFO를 두면 되지 않나?

8장. 하드웨어가 커지고, L0의 뱅크 규칙에서는 무한 버퍼여도 최대 6.5%로, 하드웨어 변경 없는 skew(9.2%)보다 작다. 둘을 합치면 더 좋을 수 있으므로 보완 관계다.

Q3. 왜 50 MHz인가? 100 MHz면?

보드 프리셋이 50 MHz다. 비트스트림은 100 MHz 타이밍을 만족한다. 사이클 수는 DMA가 따라오면 클럭과 무관하지만, 100 MHz에서 DMA가 따라오는지는 재지 않았다. 전체 시간은 7.5절(50 MHz 기준).

Q4. 디노이저 말고 다른 망에서도 되나?

구조가 다른 디노이저 열 개(채널 32, 45, 64, maxout, 평균)와 CIFAR-10 분류기에서 같은 순서였다(7.4절). 그리고 어떤 층에서 이득이 가능한지는 조건 (2)로 데이터 없이 판정할 수 있다.

Q5. 보드 절감(8.13%)과 count(9.20%)가 왜 다른가?

영상 묶음이 다르다(4장 대 79장). 같은 영상에서는 count 절감과 보드 절감이 같다(05.png의 L2: 둘 다 8.644%).

Q6. 예측이 틀린 네 층은 무엇인가?

모두 conv8, +1사이클. 마지막 창의 계산이 30사이클인 경우로, 규칙을 읽어낸 실행에 없던 경우다. 실행 뒤에 원인을 찾았고 예측은 고치지 않았다.

Q7. 정수 출력이 정말 같은가?

재배치는 채널별 연산(ReLU, 재양자화)만 거치고 같은 곱을 더한다. skew는 덧셈 순서만 바꾸고 넘침이 없다($|\text{bias}| + K \cdot 127 \cdot 128 < 2^{31}$). 실제로 80장 출력, 모든 창의 누산값, 보드의 층 출력에서 확인했다(5.6절).

Q8. 왜 W = 4에서만 효과가 있나?

32출력 층은 W = 1, 2에서 적재가 계산보다 늘 길어서(조건 (2) 성립) 배치가 영향을 줄 수 없다. 64출력 층(B)과 45출력 층(B-45)은 W = 2에서도 이득이 있다.

Q9. VARA와는 무엇이 다른가?

VARA는 ReRAM 가속기에서 0인 입력끼리 **모아** 통째로 건너뛸 블록을 만들고, 실행 중 인덱스 유닛으로 읽는다. 우리는 0이 아닌 값을 뱅크에 **흩어** 균형을 맞추고, 순열을 가중치에 접어 인덱스가 필요 없다. 그리고 skew는 데이터조차 필요 없다.

Q10. 미리 알던 것이 있는데 사전등록이 의미 있나?

알던 것은 모두 VI장에 밝혔다(10.2절). 1차 프로토콜의 가설 H1–H3, 2차의 H4–H6은 미리 알지 못한 망과 영상, W에까지 걸쳐 있고, 모두 성립했다. 보드의 L2·L3는 그 전에 보드에서 돌린 적이 없다.

Q11. 채널 수가 4의 배수가 아니면?

B-45(45채널)에서는 기존 순서가 이미 skew와 같다((45s + p) mod 4 = (s + p) mod 4). 그래서 L2 = L0, L3 = L1이고, 재배치(L1)만 이득을 준다(W = 4에서 1%대).

Q12. 에너지는?

재지 않았다. 사이클과 호스트 시간만 쟀다.

Q13. 왜 L1′도 보드에서 돌렸나?

재배치에 데이터(3장 × 잡음 2번)와 탐색(시작점 5개)을 더 주면 skew를 따라잡는지 보려는 것이다. L1′(8.10%)은 L1(7.46%)보다 낫지만 skew(9.20%)에는 못 미쳤고, 보드에서도 네 영상 모두 L2보다 사이클이 많았다.

12. 기호표와 용어집

기호	뜻	보드 망의 값
K	창의 탭 수 = 9N (3×3 커널)	288 (conv2–conv9)
N	층의 입력 채널 수	32
COUT	층의 출력 채널 수	32 (마지막 층 4)
P	출력 레인 수	16
T	탭 뱅크 수	4
W	창 포트가 사이클당 받는 값 수	보드 4
G = ⌈COUT/P⌉	출력 그룹 수	2
m	창에서 0 아닌 탭이 가장 많은 뱅크의 탭 수	conv5 평균 47.6 (L0)
n	창의 0 아닌 탭 수	
s, p	3×3 위치(0–8), 채널(0–N−1)	
j	슬롯 번호, 뱅크 = j mod 4	
π	재배치 순열: 위치 p에 원래 채널 π(p)	
class	채널 자리 p mod 4 (L0에서 같은 뱅크를 쓰는 채널 묶음)	
g	마지막 출력 그룹의 출력 수(채움/비움 식)	16

용어	뜻
탭 / 창 / 슬롯	창의 값 하나 / 출력 하나에 필요한 K개 값 / 창 안의 자리(뱅크를 정함)
lockstep	네 뱅크가 함께 다음 일로 넘어가는 방식
배치(layout)	어느 탭을 어느 슬롯에 두는가. L0 기존, L1/L1′ 재배치, L2 skew, L3 skew + 재배치
접기(folding)	재배치를 앞뒤 층 가중치에 미리 반영해 실행 중 추가 작업을 없애는 것
재양자화	INT32 합을 다음 층의 7비트 입력으로 줄이는 것(채널마다 스케일)
QAT	양자화를 고려한 학습
count	식 (1)을 층의 모든 창에 대해 더한 사이클 수(호스트 에뮬레이션)
채움/비움, 예측 사이클	층마다 붙는 짧은 추가 시간 / count + 채움/비움
CYCLES, IN_STALL	보드의 사이클 카운터, DMA 입력 대기 카운터
합산(pooled) 절감	79장의 사이클을 다 더한 뒤 계산한 절감률
paired bootstrap	영상을 다시 뽑아 구간을 구하되, 두 배치를 같은 영상 묶음에서 비교
완벽 균형, share	m = ⌈n/4⌉인 이상적 경우 / 그 절감 중 얻은 비율
사전등록	실험 전에 방법과 가설을 고정해 커밋하는 것
maxout, space-to-depth	두 맵 중 큰 값을 출력 / 2×2 픽셀을 4채널로 바꾸는 것
PSNR, σ	잡음 제거 품질(dB, 클수록 좋음) / 가우시안 잡음의 세기(0–255 척도)

13. 숫자의 출처 (저장소 `bankperm/`)

내용	파일
조건 (2), 층별 이득 가능 여부	`bounds.py` → `bounds.json`
L1 열 개 망, H1–H3	`PROTOCOL.md`, `fit.py`, `evaluate.py` → `perms.json`, `results.json`
L2, L3 열 개 망, H4–H6, 711쌍, 69층	`PROTOCOL_2.md`, `evaluate_2.py` → `results_2.json`, `perms_2.json`
Table II, 기준선, 구간	`baselines.json`, `stats_2.json`, `skew_count.json`
보드 예측과 측정 (Table I, 176/180)	`board_prediction_2.json`, `board_prediction_3.json`, `board_measured_2.json`, `board_measured_3.json`
호스트 시간 (7.5절)	`host_time.json`
FIFO 상한 (8장)	`decouple_bound.py` → `decouple_bound.json`
CIFAR-10 (7.4절)	`cifar_layouts.py` → `cifar_layouts.json`, 망은 `cifar_net/`
그림	`figures/fig_system.py`, `fig1_layouts.py`, `fig2_networks.py`, 이 문서의 그림은 `explainer/diagrams.py`
RTL	`rtl/banked_window_mac.sv` (저장소 최상위)

각 문장이 어느 파일의 어느 값에서 왔는지는 `bankperm/README.md`의 표 "Measured, counted, predicted"에 한 줄씩 있다.